

# Web Hacking Guide

---



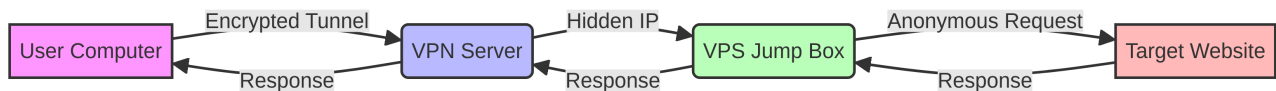
# Table of Contents

---

1. [Introduction and Basic Setup](#)
  2. [Reconnaissance \(Recon\)](#)
  3. [Asset Discovery](#)
  4. [Business Logic Vulnerabilities](#)
  5. [Modern Web Application Exploitation](#)
  6. [Post-Exploitation](#)
  7. [Conclusion and Alternative Paths](#)
-



# Introduction and Basic Setup



Web တစ်ခုကို စတင် မလေ့လာခင်မှာ ကိုယ့်ရဲ့ တကယ့် **IP Address** နဲ့ တည်နေရာကို ဖုံးကွယ်ထားဖို့က အရေးကြီးဆုံး လုပ်ဆောင်ချက် ဖြစ်ပါတယ်။ လက်တည့်စမ်းသပ်တာပဲဖြစ်ဖြစ်၊ ရည်ရွယ်ချက်ရှိရှိ လုပ်ဆောင်တာပဲ ဖြစ်ဖြစ် ဒီ အချက်တွေကို လုပ်ဆောင်ထားသင့်ပါတယ်။

## ၁။ VPN (Virtual Private Network) အသုံးပြုခြင်း

ဒါကတော့ အခြေခံအကျဆုံး အဆင့်ပါ။ ကိုယ့်ရဲ့ စက်နဲ့ အင်တာနက်ကြားမှာ **Encrypted Tunnel** တစ်ခု ခံထား ခြင်းအားဖြင့် ကိုယ့်ရဲ့ တကယ့် IP ကို ဖုံးကွယ်ထားရပါတယ်။ ဒါ့အပြင် တချို့သော **Public Tools** တွေ သုံးတဲ့အခါ မှာလည်း ISP ကနေ ခြေရာခံတာမျိုး မရှိအောင် ကာကွယ်ပေးပါတယ်။

## ၂။ VPS (Virtual Private Server) ပေါ်က လုပ်ဆောင်ခြင်း

ကျွန်တော်တို့က ကိုယ့်အိမ်က ကွန်ပျူတာကနေ တိုက်ရိုက် လုပ်ဆောင်တာထက် VPS တစ်ခုကို **“Jump Box”** အဖြစ် သုံးတာက ပိုပြီး အန္တရာယ်ကင်းပါတယ်။ **Target** ဘက်ကနေ ပြန်ပြီး ခြေရာခံ (Trace) လိုက်ရင်တောင် VPS ရဲ့ IP ကိုပဲ မြင်ရမှာ ဖြစ်ပါတယ်။ ဒါ့အပြင် VPS ပေါ်မှာ Tools တွေကို ၂၄ နာရီပတ်လုံး မောင်းနှင်ထားလို့ ရသလို၊ မြန်နှုန်းမြင့်တဲ့ **Connection** ကိုလည်း ရရှိစေပါတယ်။

## ၃။ Anonymity Tools များကို ပေါင်းစပ်ခြင်း

VPN နဲ့ VPS ကို သုံးရုံတင် မကဘဲ လိုအပ်ရင် **Proxychains** သို့မဟုတ် **Tor network** တွေကိုပါ ထပ်ဆင့် ခံထားသင့်ပါတယ်။ ဒီလို လုပ်ဆောင်ခြင်းအားဖြင့် ကိုယ့်ရဲ့ ခြေရာလက်ရာ (Logs) တွေကို လိုက်ရ ခက်သွားအောင် ပြုလုပ်နိုင်ပါတယ်။



# Reconnaissance (Recon)

Web တစ်ခုကို စပြီး ထိုးဖောက်တွေ့မယ်ဆိုရင် အရင်ဆုံး အဲဒီ Web အကြောင်းကို အရင်ဆုံး သိအောင် လုပ်ရပါတယ်။ ဒါကို **Reconnaissance (Recon)** လို့ ခေါ်တာပါ။ အခုခေတ်မှာက အရင်ကလို Tool တွေနဲ့ တန်းပြီး Scan ဖတ်တာထက် အဲဒီ Web ဘယ်လို တည်ဆောက်ထားလဲဆိုတာကို အရင် သိဖို့က ပိုအရေးကြီးပါတယ်။ ကျွန်တော် လက်တွေ့လုပ်ဆောင်တဲ့အခါမှာ သုံးတဲ့ အဆင့်တွေကို sharing လုပ်ပေးချင်ပါတယ်။

## ၁။ Global OSINT Platform များကို အသုံးပြုခြင်း

Target ဆီကို တိုက်ရိုက် မသွားခင်မှာ အချက်အလက်တွေကို အပြင်ကနေ အရင်ဆုံး လှမ်းကြည့်ရပါတယ်။ ဒီနေရာမှာ ကျွန်တော်တို့က **Shodan, Censys** တို့လို Platform တွေကို သုံးရပါတယ်။

- **Shodan.io & Search.censys.io:** ဒီ Website တွေက အင်တာနက်ပေါ်မှာရှိတဲ့ IP တွေနဲ့ Domain တွေကို အမြဲ Scan ဖတ်ပေးနေတာပါ။ ကျွန်တော်တို့က Target ရဲ့ **SSL Certificate** အချက်အလက်တွေကို ရှာကြည့်ခြင်းအားဖြင့် သူတို့ ဖျောက်ထားတဲ့ IP တွေနဲ့ Server အဟောင်းတွေကို ရှာတွေ့နိုင်ပါတယ်။
- **Fofa.info & Zoomeye.org:** ဒီကောင်တွေကတော့ အာရှဘက်က Target တွေနဲ့ တခြား Platform တွေမှာ ရှာမတွေ့တဲ့ **Tech Stack** တွေကို ရှာတဲ့အခါမှာ ပိုပြီးတော့ အသုံးဝင်ပါတယ်။

ဒီ Platform တွေကို သုံးခြင်းအားဖြင့် ကျွန်တော်တို့က Target ဆီကို တစ်ချက်မှ လှမ်းမခေါက်ဘဲနဲ့ သူတို့ သုံးထားတဲ့ IP Range တွေ၊ ပွင့်နေတဲ့ Port တွေနဲ့ Service ဗားရှင်းတွေကို အသံတိတ် သိနိုင်ပါတယ်။

## ၂။ Infrastructure & Cloud Fingerprinting

နောက်တစ်ချက်က Target က ဘယ် Cloud ပေါ်မှာ ရှိနေတာလဲဆိုတာကို သိဖို့ပါ။ သူတို့က **AWS** လား၊ **Azure** လား ဒါမှမဟုတ် **Google Cloud (GCP)** လား? အဲဒါကို သိမှသာ ကျွန်တော်တို့က ဘယ်လို Payload မျိုးတွေ သုံးလို့ရမလဲဆိုတာ စဉ်းစားလို့ ရလာတာပါ။ ဥပမာ- AWS သုံးထားတယ်ဆိုရင် **SSRF** လိုမျိုး အားနည်းချက်ကနေ တစ်ဆင့် Cloud Metadata တွေကို လှမ်းခေါ်လို့ ရမလားဆိုတာကို ကြည့်လို့ရပါတယ်။

## ၃။ Browser Extension များဖြင့် အချက်အလက် စုဆောင်းခြင်း

Web ကို Browser နဲ့ ကြည့်နေရင်းနဲ့တင် အချက်အလက်တွေ အများကြီး ရနိုင်ပါတယ်။

- **Wappalyzer:** ဒါကတော့ Target က ဘာ Technology တွေ သုံးထားသလဲ (ဥပမာ- PHP လား၊ Node.js လား၊ ဘယ်လို WAF မျိုး သုံးထားသလဲ) ဆိုတာကို ချက်ချင်း သိစေပါတယ်။
- **jsrecon:** ဒါက အရမ်း အသုံးဝင်ပါတယ်။ Web တစ်ခုထဲမှာ ရှိနေတဲ့ JavaScript ဖိုင်တွေ အားလုံးကို လိုက်ဖတ်ပြီး အဲဒီထဲမှာ ပုန်းနေတဲ့ **Hidden API Endpoints** တွေနဲ့ **API Route** တွေကို သူ့အလိုလို ဖော်ထုတ်ပေးပါတယ်။

## ၄။ JavaScript ဖိုင်များကို ခွဲခြမ်းစိတ်ဖြာခြင်း

ကျွန်တော်တို့က JavaScript ဖိုင်တွေ (ဥပမာ- main.js, chunk.js) ကို သေချာ ဖတ်ကြည့်ဖို့ လိုအပ်ပါတယ်။ Tool တွေက Endpoint တွေကိုပဲ ပြနိုင်ပေမယ့် ကျွန်တော်တို့ကတော့ အဲဒီ Code ထဲက Logic တွေကို ဖတ်ပြီး “ဒီ Function က ဘယ်လို အလုပ်လုပ်သလဲ” ဆိုတာကိုပါ သိအောင် လုပ်ဆောင်ရပါတယ်။ ဒီထဲမှာတင် Developer တွေ မေးပြီး ထားခဲ့တဲ့ **API Keys** တွေနဲ့ Documentation ထဲမှာ မပါတဲ့ လုပ်ဆောင်ချက်တွေကို ရှာတွေ့နိုင်ပါတယ်။

## ၅။ Business Logic Flow ကို နားလည်အောင် လုပ်ခြင်း

ဒါကတော့ Web ရဲ့ လုပ်ဆောင်ချက် Flow တစ်ခုလုံးကို မြေပုံဆွဲတာပါ။ User တစ်ယောက်က Register လုပ်တာ ကနေ စပြီး ငွေပေးချေတာအထိ အဆင့်ဆင့် ဘယ်လိုသွားသလဲဆိုတာကို ကြည့်ရတာပါ။ ဒီလို ကြည့်ရင်းနဲ့မှ “ငါ ဒီ နေရာမှာ **Parameter** တစ်ခုကို ပြောင်းလိုက်ရင် ဘာဖြစ်မလဲ?” ဆိုတဲ့ Logic အမှားတွေကို ရှာတွေ့လာတာ ဖြစ်ပါတယ်။

*အချုပ်ကတော့ Recon လုပ်တယ်ဆိုတာ Target ရဲ့ တံခါးကို လိုက်ခေါက်တာ မဟုတ်ဘဲ အပြင်ကနေ အဆောက်အအုံ ပုံစံကို လှမ်းကြည့်နေတာ ဖြစ်ပါတယ်။ အသံတိတ်လေး၊ ပိုပြီးတော့ ထိရောက်လေပါပဲ။*



# Asset Discovery

Target တစ်ခုကို ဖောက်ထွင်းဖို့ ကြိုးစားတဲ့အခါ ပင်မ Website (target.com) တစ်ခုတည်းကိုပဲ ကြည့်နေရင် အလုပ်မဖြစ်ပါဘူး။ ပင်မ web တွေကလဲ လုံခြုံရေးကို အရမ်းတင်းထားလေ့ရှိပါတယ် အခုခေတ် ကုမ္ပဏီကြီးတွေမှာ မေ့ကျန်နေတဲ့ **Subdomains** တွေ၊ စမ်းသပ်ဆဲ Server တွေနဲ့ အကာအကွယ် သိပ်မရှိတဲ့ **Cloud Assets** တွေ အများကြီး ရှိနေတတ်ပါတယ်။ ဒါတွေကို အကုန်လိုက်ရှာတာကို **Asset Discovery** လုပ်တယ်လို့ ခေါ်ပါတယ်။

## ၁။ Subdomain Enumeration

ပင်မ Web ထက်စာရင် Subdomain တွေမှာ အားနည်းချက် ရှိဖို့ ပိုများပါတယ်။ ဥပမာ- dev.target.com သို့မဟုတ် staging.target.com လိုမျိုး နေရာတွေဟာ Developer တွေ စမ်းသပ်ဖို့ လုပ်ထားတာ ဖြစ်တဲ့ အတွက် Security အားနည်းနေတတ်ပါတယ်။

ကျွန်တော်တို့က အဲဒီ Subdomain တွေကို ရှာဖွေတဲ့အခါ-

- **Passive Discovery:** တိုက်ရိုက် Scan မဖတ်ဘဲ **Certificate Transparency Logs** တွေကို ကြည့်ပြီး ရှာဖွေရပါတယ်။ ဒီနေရာမှာ crt.sh လို Website မျိုးကို သုံးလို့ရပါတယ်။
- **Active Discovery:** Wordlist တွေကို သုံးပြီး Subdomain ရှိမရှိ လိုက်စစ်တာမျိုးပါ။ ဒါပေမဲ့ အလွန်အကျွံလုပ်ရင် Firewall က သိသွားနိုင်တဲ့အတွက် သတိထားပြီး လုပ်ဆောင်ရပါတယ်။

## ၂။ Shadow IT Discovery

တစ်ခါတလေမှာ ကုမ္ပဏီတွေက သူတို့ရဲ့ Project အသေးလေးတွေကို main server ပေါ်မှာ မတင်ဘဲ **Netlify, Vercel, Heroku** လိုမျိုး Third-party platform တွေပေါ်မှာ တင်ထားတတ်ပါတယ်။ ဒါတွေကို ကျွန်တော်တို့ က လိုက်ရှာရပါတယ်။ ဒါဟာ Target ရဲ့ တရားဝင် Infrastructure ထဲမှာ မရှိပေမယ့် အဲဒီကနေတစ်ဆင့် ပင်မ System ရဲ့အချက်အလက်တွေကို ရသွားနိုင်ပါတယ်။

## ၃။ Port & Service Analysis

ဘယ် Subdomain တွေ ရှိသလဲဆိုတာ သိပြီဆိုရင် အဲဒီ Subdomain တစ်ခုချင်းစီမှာ Port တွေ ပွင့်နေသလဲ ဆိုတာကို ကြည့်ရပါတယ်။ **Port 80 (HTTP)** နဲ့ **443 (HTTPS)** အပြင် တခြား Port တွေ ပွင့်နေသလားဆိုတာ

ကို စစ်ဆေးရပါတယ်။

ကျွန်တော်တို့က Port ပွင့်နေရုံတင် မဟုတ်ဘဲ အဲဒီနောက်ကွယ်မှာ ဘာ Service ဗားရှင်းတွေ သုံးထားသလဲဆိုတာကိုပါ စစ်ဆေးရပါတယ်။ ဥပမာ- အဟောင်းဖြစ်နေတဲ့ **FTP Service** တွေ အကာအကွယ်မရှိတဲ့ **Database Port** တွေ ပွင့်နေရင် အဲဒါကနေတစ်ဆင့် ဝင်ရောက်ဖို့ အခွင့်အလမ်း ပိုများစေပါတယ်။

## ၄။ Virtual Host (VHost) Discovery

တစ်ခါတလေမှာ IP Address တစ်ခုတည်းပေါ်မှာတင် Web ပေါင်းများစွာကို တင်ထားတတ်ပါတယ်။ ကျွန်တော်တို့က IP တစ်ခုကို သိပြီဆိုရင် အဲဒီ IP ပေါ်မှာ တခြား ဘယ် Domain တွေ ရှိနေသေးလဲဆိုတာကိုပါ ရှာဖွေရပါတယ်။ ဒီလိုရှာဖွေခြင်းအားဖြင့် Target ရဲ့ ပင်မစာမျက်နှာမှာ မမြင်ရတဲ့ **Admin Panel** တွေ **Internal Tool** တွေကို ရှာတွေ့သွားတတ်ပါတယ်။

ကျွန်တော်ကတော့ Target ရဲ့ Subdomain တစ်ခုကို တွေ့ပြီဆိုရင် အဲဒီ Subdomain က ဘယ် IP ပေါ်မှာ ရှိသလဲ၊ အဲဒီ IP က ဘယ် Cloud Provider (ဥပမာ- AWS) ရဲ့ Range ထဲမှာလဲဆိုတာကို အရင်ကြည့်ပါတယ်။ ပြီးရင် အဲဒီ Subdomain ထဲက JavaScript တွေကို အပေါ်မှာပြောခဲ့တဲ့ **jsrecon** လိုမျိုးနဲ့ ထပ်ပြီး အချက်အလက် နှိုက်ပါတယ်။

ရည်ရွယ်ချက်က Target ကို မျက်နှာစာမျိုးစုံကနေ မြင်အောင်ကြည့်ဖို့ပါပဲ။ မြင်ကွင်း ကျယ်လေလေ၊ ဖောက်ထွင်းဖို့ အားနည်းချက် တွေဖို့ အခွင့်အလမ်း ပိုများလေလေ ဖြစ်ပါတယ်။

**Asset Discovery** အပိုင်းကတော့ ဒီလောက်ပါပဲ။ အခုဆိုရင် ကျွန်တော်တို့မှာ တိုက်ခိုက်လို့ရမယ့် နေရာတွေရဲ့ မြေပုံ တစ်ခု ရလာပြီလို့ ပြောလို့ရပါတယ်။

---



# Business Logic Vulnerabilities

(လုပ်ဆောင်ချက်ထဲက အားနည်းချက်များ)

**Business Logic** ဆိုတာ Web တစ်ခုက သတ်မှတ်ထားတဲ့ စည်းမျဉ်းတွေအတိုင်း အလုပ်လုပ်အောင် လုပ်ဆောင်ထားတာကို ခေါ်တာပါ။ ဥပမာ- ပစ္စည်းတစ်ခုကို ဝယ်မယ်ဆိုရင် အရင်ဆုံး ငွေပေးချေရမယ်၊ ပြီးမှ ပစ္စည်းပို့တဲ့ အဆင့်ကို ရောက်မယ် ဆိုတာမျိုးပေါ့။ ကျွန်တော်တို့က အဲဒီ စည်းမျဉ်းတွေကို ကျော်ဖြတ်ပြီး စနစ်ကို ကိုယ်ဖြစ်စေချင်သလို ကစားတာကို **Business Logic Attack** လို့ ခေါ်ပါတယ်။

## ၁။ Workflow Analysis ကို အရင်လုပ်ဆောင်ရပါတယ်

Web တစ်ခုရဲ့ လုပ်ဆောင်ချက် အဆင့်ဆင့် ကို အရင်ဆုံး သေချာ နားလည်အောင် လုပ်ရပါတယ်။ ဥပမာ- Account Password Reset လုပ်တဲ့အခါ-

1. Email ရိုက်ထည့်တယ်။
2. OTP ကုဒ် ပို့ပေးတယ်။
3. OTP ရိုက်ထည့်တယ်။
4. Password အသစ် ပြောင်းတယ်။

ကျွန်တော်တို့က ဒီအဆင့်တွေကြားထဲမှာ “အဆင့်ကျော်လို့ ရမလား” ဒါမှမဟုတ် တစ်ဆင့်နဲ့ တစ်ဆင့်ကြားမှာ Parameter တွေကို ပြောင်းလဲလို့ ရမလား ဆိုတာကို စမ်းသပ်ရပါတယ်။ ဥပမာ- အဆင့် ၃ ကို မသွားဘဲ အဆင့် ၄ ကို တိုက်ရိုက် သွားလို့ရနေရင် အဲဒါဟာ Logic အမှားပဲ ဖြစ်ပါတယ်။

## ၂။ IDOR (Insecure Direct Object Reference)

ဒါကတော့ အဖြစ်အများဆုံး အားနည်းချက်ပါ။ Web က User တစ်ယောက်ချင်းစီရဲ့ Data တွေကို ID နံပါတ်တွေနဲ့ ခွဲခြားထားတတ်ပါတယ်။ ကျွန်တော်တို့က ကိုယ့်ရဲ့ Data ကို ကြည့်နေတဲ့အချိန်မှာ Browser က ပို့လိုက်တဲ့ Request ထဲက ID နံပါတ်ကို တခြားသူရဲ့ ID နဲ့ လဲကြည့်တာမျိုးပါ။

ဥပမာ- <https://target.com/api/user/101> ဆိုတဲ့နေရာမှာ 101 ကို 102 လို့ ပြောင်းလိုက်တဲ့အခါ တခြား User ရဲ့ အချက်အလက်တွေကို မြင်ရတယ်ဆိုရင် ဒါဟာ **Access Control** စနစ်ရဲ့ Logic အမှား ဖြစ်ပါ



တယ်။ ဒါကို ရှာဖွေဆိုင်ရာ Tool တွေထက် ကိုယ်တိုင် Parameter တွေကို လိုက်ပြောင်းကြည့်တာက ပိုထိရောက်ပါတယ်။

## ၃။ ငွေပေးချေမှုနှင့် ဈေးနှုန်းဆိုင်ရာ Logic အမှားများ

ဒီအပိုင်းကတော့ စီးပွားရေးလုပ်ငန်းတွေအတွက် အရမ်း အန္တရာယ်များပါတယ်။ ကျွန်တော်တို့ စစ်ဆေးရမယ့် အချက်တွေကတော့-

- **Price Manipulation:** ပစ္စည်းတစ်ခုကို ဝယ်ယူတဲ့ Request ပို့တဲ့အခါ ဈေးနှုန်း နေရာမှာ တန်ဖိုးကို လျှော့ချကြည့်တာမျိုးပါ။ (ဥပမာ- 100တန်ပစ္စည်းကို 1 လို့ ပြောင်းကြည့်တာမျိုး)။
- **Negative Values:** ပစ္စည်းအရေအတွက်နေရာမှာ Minus တန်ဖိုး (ဥပမာ- -1) ထည့်လိုက်တဲ့အခါ စုစုပေါင်း ကျသင့်ငွေက လျော့သွားမလားဆိုတာမျိုး စစ်ဆေးရပါတယ်။
- **Coupon Stacking: Discount Coupon** တွေကို တစ်ခုထက်မက သုံးလို့ရနေမလား၊ ဒါမှမဟုတ် သုံးပြီးသား Coupon ကိုပဲ ထပ်ခါတလဲလဲ သုံးလို့ရနေမလားဆိုတာကို စမ်းသပ်တာ ဖြစ်ပါတယ်။

## ၄။ OTP နှင့် Authentication Logic

အခုခေတ်မှာ MFA တွေ သုံးလာကြပေမယ့် အဲဒီနေရာမှာလည်း Logic အမှားတွေ ရှိနေတတ်ပါတယ်။

- **Rate Limiting Bypass:** OTP ရိုက်ထည့်တဲ့အခါ အကြိမ်ရေ ကန့်သတ်ချက် မရှိရင် Brute Force လုပ်ပြီး ဝင်လို့ ရသွားနိုင်ပါတယ်။
- **Response Manipulation:** OTP မှားရိုက်ထည့်လိုက်ပေမယ့် Server က ပြန်ပို့တဲ့ “Invalid Code” ဆိုတဲ့ Response ကို “Success” လို့ Client ဘက်ကနေ ပြောင်းလိုက်ရင် စနစ်ထဲ ပေးဝင်မလားဆိုတာမျိုးကို ကျွန်တော်တို့က Burp Suite နဲ့ စမ်းသပ်ရပါတယ်။

Web တစ်ခုကို စမ်းသပ်တဲ့အခါ “ဒီစနစ်ကို တည်ဆောက်တဲ့သူက ငါ့ကို ဘာလုပ်စေချင်တာလဲ?” ဆိုတာကို အရင် စဉ်းစားပါတယ်။ ပြီးမှ “သူ မစဉ်းစားထားတဲ့ လမ်းကြောင်းကနေ ငါ ဘယ်လို သွားလို့ရမလဲ?” ဆိုတာကို လိုက်ရှာတာပါ။

Business Logic Attack တွေမှာ အောင်မြင်ဖို့ဆိုရင် စနစ်တစ်ခုလုံးရဲ့ အလုပ်လုပ်ပုံကို စေ့စေ့စပ်စပ် သိနေဖို့ လိုပါတယ်။ အသံတိတ် စောင့်ကြည့်ပြီး စနစ်ရဲ့ လိုဟာချက်ကို ရှာဖွေတာက တကယ့် ထိရောက်တဲ့ နည်းလမ်းပါပဲ။

---



# Modern Web Application Exploitation

Modern Web Application တွေဟာ အခုခေတ်မှာ **WAF (Web Application Firewall)** တွေနဲ့ ခံစစ် ကောင်းကောင်း ပြင်ဆင်လာကြပါတယ်။ ဒါကြောင့် ကျွန်တော်တို့က အရင်ကလို **Payload** အဟောင်းတွေကို သုံး လို့မရတော့ဘဲ စနစ်ရဲ့ အလုပ်လုပ်ပုံကို ကြည့်ပြီး ပိုမိုနက်နဲတဲ့ နည်းလမ်းတွေကို သုံးရပါတယ်။

## ၁။ Modern SQL Injection (Blind & Time-based)

အခုခေတ် Web တွေက **ORM** တွေ သုံးလာတဲ့အတွက် ရိုးရိုး SQL Injection တွေ တွေ့ဖို့ ခဲယဉ်းသွားပါပြီ။ ဒါပေမဲ့ Developer တွေက **Raw Query** တွေကို မှားသုံးမိတဲ့အခါမျိုးမှာ အားနည်းချက် ရှိနေတတ်ပါတယ်။

- **Blind SQLi:** Server က Error မပြတဲ့အခါမှာ “မှန်တယ်/မှားတယ်” ဆိုတဲ့ တုံ့ပြန်မှု (**Boolean response**) ကို ကြည့်ပြီး Database ထဲက Data တွေကို တစ်လုံးချင်းစီ နှိုက်ယူရပါတယ်။
- **Time-based SQLi:** ဒါကတော့ ပိုပြီး အသံတိတ်ပါတယ်။ ကျွန်တော်တို့ ပို့လိုက်တဲ့ Request ကို Server က ၅ စက္ကန့်လောက် ကြာမှ ပြန်ပို့အောင် လုပ်ဆောင်ပြီး အဲဒီကြာချိန်ကို ကြည့်ကာ Database အချက်အလက်တွေကို ခွဲခြားရတာ ဖြစ်ပါတယ်။

## ၂။ Remote Code Execution (RCE) - စနစ်တစ်ခုလုံးကို ထိန်းချုပ်ခြင်း

**RCE** ဆိုတာကတော့ Web Server ပေါ်မှာ ကျွန်တော်တို့ စိတ်ကြိုက် Command တွေ ရိုက်ထည့်လို့ရသွားတဲ့ အဆင့်ပါ။ ဒါဟာ အန္တရာယ်အရှိဆုံး အားနည်းချက်ပဲ ဖြစ်ပါတယ်။

- **File Upload Vulnerability:** ဓာတ်ပုံတင်တဲ့နေရာမျိုးမှာ PHP သို့ ကိုယ်ဖြစ်စေချင်တဲ့ Script ဖိုင် တွေကို **Extension Bypass** လုပ်ပြီး တင်ကြည့်တာမျိုးပါ။
- **Insecure Deserialization:** App က လက်ခံလိုက်တဲ့ Data တွေကို ပြန်ပြီး Object အဖြစ် ပြောင်း တဲ့အခါမှာ Logic အမှားရှိနေရင် ကျွန်တော်တို့က ကိုယ်ပိုင် Code တွေကို ထည့်သွင်း အလုပ်လုပ်ခိုင်းလို့ ရ သွားတတ်ပါတယ်။

## ၃။ Server-Side Request Forgery (SSRF)

ဒါကတော့ **Cloud Infrastructure** နဲ့ တိုက်ရိုက် ပတ်သက်ပါတယ်။ **SSRF** ဆိုတာ Web Server ကို သုံးပြီး သူတို့ရဲ့အတွင်းပိုင်း Network (Internal Network) ထဲကို လှမ်းပြီး Request ပို့ခိုင်းတာပါ။

အထူးသဖြင့် Cloud သုံးထားတဲ့ Target တွေမှာဆိုရင် **169.254.169.254** ဆိုတဲ့ **Metadata Endpoint** ကို လှမ်းခေါ်ကြည့်ရပါတယ်။ အဲဒီကနေတစ်ဆင့် Cloud ရဲ့ **Access Keys** တွေနဲ့ လျှို့ဝှက် အချက်အလက်တွေ ကို ရသွားနိုင်ပါတယ်။ ဒီနေရာမှာ ကျွန်တော်တို့က Web ရဲ့ URL parameter တွေကို ကစားပြီး စမ်းသပ်ရပါတယ်။

## SSRF Exploitation ဥပမာ- AWS Metadata ကို ရယူခြင်း

Cloud ပေါ်မှာ ရှိတဲ့ Web Server တွေဟာ သူတို့ရဲ့ စက်နဲ့ ပတ်သက်တဲ့ လျှို့ဝှက် အချက်အလက် (**Credentials, Keys**) တွေကို Internal IP ဖြစ်တဲ့ 169.254.169.254 ကနေ ရယူနိုင်ပါတယ်။ SSRF အားနည်းချက်ကို တွေ့ရှိပြီဆိုရင်၊ ကျွန်တော်တို့က Web Server ကို အပြင်ကနေ URL parameter ကနေ တစ်ဆင့် `http://169.254.169.254/latest/meta-data/` ကို ခေါ်ခိုင်းလိုက်ပါတယ်။ ဒီလို လုပ်ဆောင်ခြင်းအားဖြင့် Cloud Credentials တွေကို ရသွားနိုင်ပြီး တိုက်ခိုက်မှု အဆင့်ကို ပိုပြီး မြှင့်တင်နိုင်ပါတယ်။

## ၄။ WAF Evasion (Firewall ကို ကျော်ဖြတ်ခြင်း)

ကျွန်တော်တို့ ပို့လိုက်တဲ့ Payload တွေကို Firewall က သိမသွားအောင် လုပ်ဆောင်ဖို့ လိုပါတယ်။

- **Encoding:** Payload တွေကို **URL Encoding, Double URL Encoding** သို့ **Hex Encoding** တွေ ပြောင်းပြီး ပို့ကြည့်တာပါ။
- **Case Variation & Comments:** SQL Query တွေမှာ **SeLeCt** လို့ ရေးတာမျိုး ဒါမှမဟုတ် `/*...*/` လိုမျိုး Comment တွေ ကြားညှပ်ပြီး Firewall ရဲ့ Filter တွေကို ရှောင်ထွေးအောင် လုပ်ဆောင်ရပါတယ်။

ကျွန်တော်ကတော့ အားနည်းချက်တစ်ခုကို စမ်းသပ်တဲ့အခါ Tool က ပြတဲ့ အဖြေကိုပဲ မကြည့်ပါဘူး။ ဥပမာ- **SQL Injection** စမ်းရင် Query တစ်ခု ပို့လိုက်လို့ Response ကြားသွားသလား၊ ဒါမှမဟုတ် စာလုံးတစ်လုံး ပြောင်းသွားသလားဆိုတာကို သေချာ စောင့်ကြည့်ပါတယ်။

Modern Exploitation မှာ အဓိကက **“Context”** ပါပဲ။ Target ရဲ့ Backend က ဘာသုံးထားလဲ၊ Database က ဘာလဲဆိုတာကို သိမှသာ အဲဒီစနစ်နဲ့ ကိုက်ညီမယ့် Payload ကို ကိုယ်တိုင် တည်ဆောက်လို့ ရမှာ ဖြစ်ပါတယ်။

---



## Post-Exploitation

---

Web vulnerability တစ်ခုခုကနေတစ်ဆင့် စနစ်ထဲကို ခြေတစ်ဖက် စလှမ်းနိုင်ပြီဆိုရင် ကျွန်တော်တို့ နောက်ထပ် လုပ်ဆောင်ရမယ့် အဆင့်တွေ ရှိပါတယ်။ ဒါကို **Post-Exploitation** လို့ ခေါ်ပါတယ်။ ဒီအဆင့်မှာ ရည်မှန်းချက် က အခွင့်အာဏာကို မြှင့်တင်ဖို့နဲ့ လိုချင်တဲ့ အချက်အလက်တွေကို ရယူဖို့ ဖြစ်ပါတယ်။

### ၁။ Privilege Escalation

ကျွန်တော်တို့ စပြီး ဝင်ခွင့်ရတဲ့ အကောင့်ဟာ အများအားဖြင့် အကန့်အသတ်ရှိတဲ့ User အကောင့် (ဥပမာ- **www-data**) မျိုးပဲ ဖြစ်တတ်ပါတယ်။ စနစ်တစ်ခုလုံးကို ထိန်းချုပ်ဖို့အတွက် Admin သို့ Root Level ရောက်အောင် လုပ်ဆောင်ရပါတယ်။

- **Vertical Escalation:** ဒါကတော့ သာမန် User ကနေ **Root (System Admin)** ဖြစ်အောင် တက်တာပါ။ System ထဲမှာရှိတဲ့ **Kernel** ဗားရှင်း အဟောင်းတွေရဲ့ အားနည်းချက်ကို သုံးတာမျိုး ဒါမှမဟုတ်

မှားယွင်းစွာ သတ်မှတ်ထားတဲ့ **SUID** ဖိုင်တွေကို ရှာဖွေပြီး အသုံးပြုတာမျိုး ဖြစ်ပါတယ်။

- **Horizontal Escalation:** ဒါကတော့ ကိုယ်နဲ့ အဆင့်တူတဲ့ တခြား User တစ်ယောက်ရဲ့ အကောင့်ထဲ ကို ကူးပြောင်းတာပါ။ အဲဒီ User ရဲ့ ဖိုင်တွေထဲမှာ သိမ်းထားတဲ့ လျှို့ဝှက်နံပါတ်တွေ သို့ **SSH Keys** တွေကို ရှာဖွေပြီး လုပ်ဆောင်ရပါတယ်။

## ၂။ Lateral Movement & Pivoting (အတွင်းပိုင်း ကွန်ရက်ထဲ ဆက်တိုးခြင်း)

Web Server တစ်ခုကို ထိန်းချုပ်နိုင်ပြီဆိုရင် အဲဒီ Server ကနေတစ်ဆင့် ကုမ္ပဏီရဲ့ အတွင်းပိုင်း Network ထဲ မှာရှိတဲ့ တခြား ကွန်ပျူတာတွေနဲ့ Database တွေကို ဆက်ပြီး ရှာဖွေရပါတယ်။

ကျွန်တော်တို့က အဲဒီ Web Server ကို “**Pivot Point**” အဖြစ် သုံးပြီး အပြင်ကနေ တိုက်ရိုက် မမြင်ရတဲ့ **Internal Services** တွေကို လှမ်းပြီး Scan ဖတ်ပါတယ်။ ဒီနေရာမှာ ကျွန်တော်တို့က **Port Forwarding** သို့မဟုတ် **Tunneling** နည်းလမ်းတွေကို သုံးပြီး အတွင်းပိုင်း Network ထဲကို လှမ်းကြည့်တာ ဖြစ်ပါတယ်။

## ၃။ Maintaining Access (စနစ်ထဲမှာ ဆက်ရှိနေအောင် လုပ်ခြင်း)

ကိုယ့်ရဲ့ ဝင်ပေါက်ကို Admin က သိသွားပြီး ပိတ်လိုက်ရင်တောင် နောက်တစ်ကြိမ် ပြန်ဝင်လို့ရအောင် ပြင်ဆင်ထား ရပါတယ်။

- **Web Shells:** Web directory ထဲမှာ ကိုယ်ပိုင် Script ဖိုင်လေးတွေ ဝှက်ထားခဲ့တာမျိုးပါ။
- **New Users/Keys:** စနစ်ထဲမှာ User အသစ် တစ်ခု တိုးထားခဲ့တာမျိုး ဒါမှမဟုတ် ကိုယ့်ရဲ့ **SSH Public Key** ကို `authorized_keys` ထဲ ထည့်ထားခဲ့တာမျိုး လုပ်ဆောင်ရပါတယ်။ (မှတ်ချက် - ဒါကို စမ်းသပ်တဲ့အခါမှာ သဲလွန်စ မကျန်အောင် သတိထားပြီး လုပ်ဆောင်ရပါတယ်)

## ၄။ Data Exfiltration (အချက်အလက် ထုတ်ယူခြင်း)

ဒါကတော့ နောက်ဆုံး ရည်မှန်းချက်ပါ။ လိုချင်တဲ့ အရေးကြီး Data တွေကို အပြင်ကို ထုတ်ယူတဲ့အခါ Firewall တွေ **IDS (Intrusion Detection System)** တွေ မိမသွားအောင် လုပ်ဆောင်ရပါတယ်။

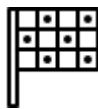
- **Encoding & Compression:** Data တွေကို **Encrypt** လုပ်ပြီး **Zip** ချုပ်တာမျိုး လုပ်ရပါတယ်။
- **Stealthy Transfer:** Data တွေကို အလုံးအရင်းနဲ့ တန်းမပို့ဘဲ နည်းနည်းချင်းစီ ခွဲပို့တာမျိုး ဒါမှမဟုတ် **DNS Tunneling** လိုမျိုး နည်းလမ်းတွေကို သုံးပြီး အချက်အလက် ထုတ်ယူရပါတယ်။

စနစ်ထဲ ရောက်သွားပြီဆိုရင် ပထမဆုံး လုပ်တာက “**Environment Survey**” ပါ။ `whoami` , `id` , `uname -a` စတဲ့ Command တွေနဲ့ ကိုယ်က ဘယ်နေရာ ရောက်နေလဲ၊ ဘယ်သူ့အဆင့်နဲ့ ရှိနေလဲဆိုတာ အရင်ကြည့်ပါ တယ်။ ပြီးရင် စနစ်ထဲမှာရှိတဲ့ **Configuration** ဖိုင်တွေ၊ ပွင့်နေတဲ့ **Internal Ports** တွေနဲ့ အသုံးပြုသူတွေရဲ့ **History** ဖိုင်တွေကို လိုက်ဖတ်ပါတယ်။

အဓိကကတော့ အလျင်စလို မလုပ်ဖို့ပါပဲ။ အသံကျယ်ကျယ် လုပ်မိလေလေ၊ စနစ်ရဲ့ Admin က သိသွားဖို့  
အခွင့်အလမ်း ပိုများလေလေ ဖြစ်လာလို့ပါ။

Post-Exploitation အပိုင်းကတော့ တကယ်ကို ကျယ်ပြန့်ပါတယ်။

---



## Conclusion and Alternative Paths

Web တစ်ခုကို အောင်မြင်စွာ ထိုးဖောက်နိုင်ပြီး Access တစ်ခု ရလာတဲ့အခါမှာ အဲဒီရလဒ်ကို ဘယ်လို အသုံးပြုမလဲဆိုတာက နောက်ဆုံးအဆင့် ဖြစ်ပါတယ်။ လက်တွေ့နယ်ပယ်မှာ ကုမ္ပဏီတွေရဲ့ တုံ့ပြန်မှုနဲ့ အခြေအနေပေါ် မူတည်ပြီး ရွေးချယ်စရာ လမ်းကြောင်းတွေ အများကြီး ရှိပါတယ်။

### ၁။ Vulnerability Reporting နှင့် Silent Fix ပြဿနာ

အသုံးအများဆုံး နည်းလမ်းကတော့ တွေ့ရှိထားတဲ့ အားနည်းချက်ကို ကုမ္ပဏီထံ Report ပို့ခြင်း ဖြစ်ပါတယ်။ ဒါပေမဲ့ လက်တွေ့မှာတော့ စိန်ခေါ်မှုတွေ ရှိပါတယ်။

- ကုမ္ပဏီအချို့ဟာ ပေးပို့လာတဲ့ Report ကို လက်ခံရရှိကြောင်း အကြောင်းမပြန်ဘဲ အသံတိတ် ပြုပြင်လိုက်တာမျိုးတွေ လုပ်ဆောင်တတ်ပါတယ်။
- **Response Lack:** တရားဝင် Bug Bounty Program မရှိတဲ့ ကုမ္ပဏီတွေမှာ Report ကို လျစ်လျူရှုထားတာမျိုး ဒါမှမဟုတ် ဥပဒေကြောင်းအရ ပြန်လည် ခြိမ်းခြောက်တာမျိုးတွေလည်း ကြုံတွေ့ရနိုင်ပါတယ်။

### ၂။ Initial Access Brokering (IAB)

Access ရရှိထားတဲ့ အဆင့်မှာတင် ရပ်တန့်ပြီး အဲဒီဝင်ပေါက် (Shell သို့ Admin Access) ကို ပြန်လည် ရောင်းချတဲ့ နည်းလမ်း ဖြစ်ပါတယ်။

- **Marketplace:** Underground forum တွေမှာ ကုမ္ပဏီရဲ့ အချက်အလက်နဲ့ Access အဆင့်အတန်းအပေါ် မူတည်ပြီး ဈေးဖြတ် ရောင်းချကြပါတယ်။

### ၃။ Ransomware နှင့် Double Extortion

ဒါကတော့ လက်ရှိအဖြစ်အများဆုံး ဖြစ်ပါတယ်။ ကုမ္ပဏီက Report ကို တန်ဖိုးမထားတဲ့အခါ ဒါမှမဟုတ် တိုက်ရိုက် ငွေရှာချင်တဲ့အခါမှာ သုံးကြပါတယ်။

- **Encryption:** Server ပေါ်က Data တွေကို Encrypt လုပ်ပြီး Lock ချလိုက်ခြင်းအားဖြင့် စနစ်တစ်ခုလုံးကို ရပ်တန့်စေပါတယ်။

- **Double Extortion: Encryption** မလုပ်ခင်မှာ အရေးကြီး Data တွေကို အရင်ခိုးထုတ်ပါတယ်။ ကုမ္ပဏီဘက်က ငွေမပေးရင် အဲဒီ Data တွေကို Public ချပြမယ်လို့ ခြိမ်းခြောက်ပြီး ငွေညှစ်တဲ့ နည်းလမ်း ဖြစ်ပါတယ်။

## ၄။ Full Disclosure (လူသိရှင်ကြား ထုတ်ဖော်ခြင်း)

ကုမ္ပဏီက အားနည်းချက်ကို သိလျက်နဲ့ မပြင်တာမျိုး ဒါမှမဟုတ် ခိုးပြီး Fix တာမျိုးတွေ လုပ်တဲ့အခါမှာ သုံးတဲ့ နည်းလမ်း ဖြစ်ပါတယ်။ အားနည်းချက်ကို ဘယ်လို ရှာဖွေခဲ့တယ်ဆိုတာကို အများသိအောင် ချပြလိုက်ခြင်းအားဖြင့် ကုမ္ပဏီရဲ့ ပေါ့ဆမှုကို ဖော်ပြအရှုက်ခွဲတာ ဖြစ်ပါတယ်။

Web Hacking ဆိုတာ တံခါးတွေကို ဖွင့်တတ်ရုံတင် မဟုတ်ဘဲ ပွင့်သွားတဲ့ တံခါးတွေရဲ့ နောက်ကွယ်က ရွေးချယ်မှုတွေကိုပါ နားလည်ထားဖို့ လိုအပ်ပါတယ်။ လက်တွေ့မှာတော့-

- **White Hat** လုပ်မယ်ဆို စနစ်တကျ Report တင်မယ် (Reward ရခြင်း၊ မရခြင်းထက် တာဝန်ယူမှုကို ဦးစားပေးမယ်)။
- **Black Hat: Access** ကို ရောင်းစားမယ် ဒါမှမဟုတ် Ransomware နဲ့ စနစ်ကို ထိန်းချုပ်မယ်။

ဘယ်လမ်းကို ရွေးမလဲဆိုတာကတော့ တစ်ဦးချင်းစီရဲ့ ရည်ရွယ်ချက်နဲ့ ခံယူချက်ပေါ်မှာပဲ မူတည်ပါတယ်။